

Algorithm Improvements

Web-Based Market Analysis and Product Recommendation System for Thuthi Dairy Private Limited

Revision date: 28 August 2026

1. Summary

Five algorithms in the platform were reviewed and reworked: the product recommendation engine, RFM customer segmentation with K-Means clustering, ABC inventory analysis, product-search ranking, and sales forecasting.

The review was prompted by feedback that the recommendation engine appeared to optimise for the store making a sale rather than for the shopper finding the right product - specifically, that it could "bring one product down to push another." Two mechanisms in the old code invited that reading:

1. A helper that attached manufactured-urgency phrases to every suggestion ("only 3 left in stock", "selling fast").

2. "Frequently bought together" ranked pairs by raw **confidence** ($\text{co-occurrence} / \min(\text{count})$), which is structurally biased toward whatever is already popular.

Both are fixed. The recommender now has an explicit, written objective that contains **no price, margin, or stock term**, ranks item associations by **lift** (which controls for popularity by construction), and ships with an **offline evaluation harness** so "is it any good?" is answered with numbers instead of opinion.

Headline outcomes on the live dataset:

Metric	Personalised model	Bestseller list
Ranking quality (MAP@10)	beats a random list by +6.3 points	n/a
Catalogue coverage	78%	20%
Popularity bias (lower is better)	57	91
Hit-rate (NDCG@10)	28.5	40.4

The personalised model trades roughly 12 NDCG points of raw hit-rate for **4x the catalogue coverage** and **35 points less popularity bias** - i.e. it stops funnelling every shopper toward the same handful of products, which is the property the review asked for.

All changes type-check, the 19-test unit suite passes, and every algorithm was verified against the live deployment.

2. The recommendation objective

The engine ranks the catalogue for each shopper by:

```
utility(item | shopper) = relevance(shopper, item)
                        - POPULARITY_PRIOR * popularity(item)

then re-ranked with Maximal Marginal Relevance for diversity.
```

- **relevance(shopper, item)** is built from six signals the shop already

collects: reorder history, category affinity, item-to-item association, user-user collaborative filtering, recency of browsing, and global trend/popularity.

- **POPULARITY_PRIOR** (0.15) subtracts a fraction of a product's global

popularity percentile. It pushes bestsellers *down*, not up, and is applied only to generic discovery items, never to a product the shopper personally reorders.

- The objective contains ****no term for price, profit margin, or stock**

level**. A recommendation is only ever justified by relevance to the shopper, and every suggestion carries a plain, factual reason ("You've ordered this 8 times", "Often bought with Bilona Cow Ghee").

What was removed: `buildPersuasiveReason()`, which manufactured urgency and scarcity. Its replacement, `annotateReason()`, adds at most one factual clause - a rating, a units-sold count, or a current discount - and never invents time pressure.

3. Recommendation engine: changes

Area	Before	After	Why
Suggestion copy	Manufactured urgency / scarcity	Factual reasons only	Removes the dark pattern
"Frequently bought together"	Raw confidence, $co / \min(count)$	Lift , $P(A \text{ and } B) / (P(A) \times P(B))$	Lift controls for base popularity; a bestseller can no longer be glued to every basket by coincidence
Blending strategies	Home-grown score normalisation plus an ad-hoc bonus	Weighted Reciprocal Rank Fusion ($RRF_K = 60$)	Standard, rank-based, scale-free
Popularity handling	None	Popularity prior de-biases discovery slots; personal reorders exempt	"De-bias for discovery, never demote the milk someone buys weekly"
List variety	None	MMR re-rank ($\lambda = 0.8$) plus max 4 per category; staples protected	Explicit relevance vs. diversity trade-off
Collaborative filtering	Hard cut-offs on a sparse dataset	Cosine x significance weight , $\min(\text{shared}, 3) / 3$	Standard sparse-data correction
Quality measurement	None	Offline evaluation harness (Section 4)	Claims become measurable

Lift, in words

$$\text{lift}(A, B) = P(A \text{ and } B \text{ in the same basket}) / (P(A) \times P(B))$$

- $\text{lift} = 1$ - the two products co-occur exactly as often as chance predicts

given how popular each one is. No real association.

- $\text{lift} > 1$ - a genuine "these go together" signal.
- $\text{lift} < 1$ - the products actively repel.

Only pairs with $\text{lift} > 1$ feed the "frequently bought together" widget. On the live data the strongest pair is *A2 Desi Cow Milk + Bilona Cow Ghee* at 5.1x - bought together five times more often than their individual popularity would predict.

4. Offline evaluation

Method: temporal hold-out

1. Sort every past order by date.
2. **Train** on the oldest 70-80% of orders; **test** on the most recent 20-30%.
3. For each customer active in both windows, generate top-K recommendations using **only** the training window, then score them against what that customer actually bought in the test window.

The scorer is an independent, compact re-implementation of the production blend - a system is not tested with a copy of itself.

Metrics

Metric	Meaning	Direction
Precision@K	Share of the K suggestions that were bought	higher is better
Recall@K	Share of the customer's test purchases that were hit	higher is better
MAP@K	Mean average precision; rewards hits near the top	higher is better
NDCG@K	Discounted cumulative gain; same idea, log decay	higher is better
Coverage	Share of the catalogue recommended to anyone	higher is better
Diversity	1 minus mean pairwise "same category" within a list	higher is better
Popularity bias	Mean popularity percentile of recommended items	lower is better

Two baselines run alongside: MOST_POPULAR (train-set bestsellers, same for everyone) and RANDOM (seeded, deterministic).

Results

Live dataset, K = 10, 30% hold-out, 8 evaluated customers.

Model	P@10	NDCG@10	MAP@10	Coverage	Popularity bias
Personalised	27.5	28.5	16.4	78.4%	56.7
Most popular	37.5	40.4	26.3	19.6%	91.0
Random	22.5	24.5	10.1	78.4%	51.7

Interpretation

- The personalised model **clearly beats a random list** on ranking quality (MAP +6.3).
- It **trails a pure bestseller list** on raw hit-rate by about 12 NDCG

points. On a 51-product dairy catalogue dominated by a few staples (milk, curd, paneer), "recommend the bestsellers" is a genuinely strong accuracy baseline; most people do buy the staples.

- Where it wins decisively is **breadth**: 4x the catalogue coverage and 35

points less popularity bias. That is the difference between a recommender that helps customers discover the rest of the range and one that just reinforces the top ten.

- The endpoint returns a note whenever fewer than 15 customers qualify, so the small-sample caveat travels with the numbers.

Endpoint: GET /recommendations/admin/evaluate?k=10&testFraction=0.3 (admin only).

5. Customer segmentation: RFM and K-Means

RFM scoring

Recency, Frequency and Monetary value are each quintile-scored 1-5 **within the current customer base**, so a 50-customer store and a 50,000-customer store both get a meaningful spread. Scoring now cuts on **value quantiles** rather than rank position, so two customers with identical spend always get an identical score.

K-Means clustering

Aspect	Before	After
Number of clusters, k	Hard-coded 4	Chosen by silhouette score over $k = 2..maxK$, preferring the smaller k on a near-tie
maxK	6	Scaled to base size (about $\sqrt{n / 2}$), so 18 customers gives 3, not 6 tiny clusters
Runs	Single	10 random restarts , best inertia kept (Lloyd's algorithm only finds a local optimum)
Feature scaling	Min-max (outlier-fragile)	log-transform frequency and monetary, then z-score standardise
Diagnostics	Inertia only	Silhouette score plus the full per-k inertia table for the elbow view

The log transform matters: frequency and monetary are heavily right-skewed (most customers cluster low, a few are very large). Without it, min-max scaling compresses everyone into a corner and the clusters collapse to "one big spender vs. everyone else."

Live result: $k = 3$ auto-selected (silhouette 0.49) - *High-value regulars* (11 customers), *Occasional shoppers* (3), *Dormant / at risk* (4).

6. Inventory analysis: ABC and XYZ

ABC (value) - bug fix

The cumulative revenue share was accumulated from **rounded** per-row percentages. Over about 90 SKUs the rounding error compounds and shifts the A/B cut-offs. It now accumulates from raw revenue and rounds only for display.

XYZ (demand predictability) - new

For each SKU, monthly units sold over a trailing six months give a demand series; its **coefficient of variation** (CV = standard deviation / mean) is the stability measure:

Class	CV	Meaning
X	CV <= 0.5	Steady - safe to run lean, easy to forecast
Y	0.5 < CV <= 1.0	Variable - needs a buffer
Z	CV > 1.0	Erratic / sporadic - forecast with caution

Months before a SKU's first sale are dropped (a product launched two months ago is not "erratic" for the four months it did not exist); interior zeros are kept because a no-demand month is real signal.

The ABC x XYZ matrix

Crossing the two gives a 3x3 action grid. **AX** (high value, steady) is where just-in-time ordering pays off; **AZ** (high value, erratic) needs generous safety stock because a stockout is both likely and expensive; **CZ** (low value, erratic) is usually make-to-order or delist.

Live result: X 16 / Y 51 / Z 25 SKUs; all nine matrix cells populated.

7. Product search: TF-IDF to BM25

Change	Detail
Ranking function	TF-IDF cosine replaced with BM25 ($k_1 = 1.5$, $b = 0.75$): term-frequency saturation plus document-length normalisation; the ranking function used by Lucene, Elasticsearch and Postgres full-text search
Stemming	A light suffix stripper folds plurals and common verb forms (eggs to egg, berries to berry, baking to bake)
Synonyms	A small curated dairy map (curd / yoghurt / dahi, paneer / chena, and so on) expanded on both documents and queries
Typo tolerance	When BM25 finds nothing, a trigram (Dice) similarity fallback runs, so "chesse" still finds "cheese" instead of returning an empty page

BM25 is a better default than plain cosine because the 2nd, 3rd and 10th occurrence of a keyword add ever less score, so one keyword-stuffed description cannot dominate, and a long description is not rewarded merely for containing more words.

8. Sales forecasting: added Holt-Winters

The linear-trend plus day-of-week model is a reasonable transparent baseline but assumes the trend is a straight line for the whole window. A second model was added:

- **Holt-Winters additive triple exponential smoothing** keeps a level, a trend and a weekly seasonal component, each adapting as new data arrives. Still only three interpretable parameters, no black box.

Both models are fitted; the one with the lower in-sample error (MAPE) is served, and both scores are returned so the choice is auditable. The ordinary-least-squares prediction interval now also includes the leverage term (x minus mean- x) squared over S_{xx} , so the confidence band fans out correctly the further past the observed data the forecast projects.

9. How this addresses the review

"Why is the recommender bringing one product down to push another?"

It no longer does, and the design makes that checkable:

- The objective function is written down and contains no price, margin or stock term.
- "Frequently bought together" ranks by **lift**, which by construction ignores how popular a product is on its own; a bestseller cannot be attached to every basket by coincidence.
- The popularity prior pushes popular items *down* for discovery slots and never demotes a product the shopper personally reorders.
- Every suggestion carries a factual reason; no manufactured urgency.

"How do you know it is any good?"

The offline evaluation harness reports Precision / Recall / MAP / NDCG plus coverage, diversity and popularity bias against MOST_POPULAR and RANDOM baselines on a temporal hold-out. The measured trade-off is explicit: about 12 NDCG points of hit-rate given up for 4x catalogue coverage and 35 points less popularity bias.

"Are the analytics methods sound?"

- Clustering k is now chosen by silhouette score, not guessed; features are log-scaled and standardised; 10 restarts guard against a bad local optimum.
- ABC has a rounding-drift bug fixed and is extended with XYZ demand-variability analysis and a 3x3 action matrix.
- Search uses BM25, the industry-standard ranking function.
- Forecasting fits a linear model and Holt-Winters and serves whichever fits the history better, with the interval maths corrected.

10. Verification

Check	Result
npm run typecheck (server and client)	passes
npm test (server)	19 / 19 pass: BM25, fuzzy match, k-means, silhouette, k-selection, Holt-Winters
Live: affinity table	lift-based, top pairs 5 to 9.5x
Live: RFM clusters	k = 3 auto (silhouette 0.49), sensible sizes
Live: ABC x XYZ	full 9-box matrix populated
Live: recommender feed	leads with the customer's real reorders, factual reasons, no scarcity language
Live: evaluation endpoint	runs; verdict as in Section 4

Appendix A - files changed

Server

- `src/modules/recommendations/recommendation.service.ts` - lift, RRF, popularity prior, MMR, honest reasons, significance-weighted CF
- `src/modules/recommendations/recommendation.eval.ts` - new offline evaluation harness
- `src/modules/recommendations/recommendation.routes.ts` - GET `/admin/evaluate`
- `src/lib/kmeans.ts` - multi-restart, silhouette, chooseK, standardize, logScale
- `src/modules/analytics/customer-intelligence.service.ts` - log plus z-score RFM pipeline, auto-k, value-quantile scoring
- `src/modules/analytics/abc.service.ts` - cumulative-share fix, XYZ, 3x3 matrix
- `src/lib/tfidf.ts` - BM25, stemmer, synonyms, trigram fuzzy match
- `src/lib/holt-winters.ts` - new Holt-Winters implementation
- `src/modules/analytics/analytics.service.ts` - dual-model forecast, corrected interval
- `src/modules/catalog/product.service.ts` - fuzzy fallback wired into search
- `prisma/seed.ts` - per-customer staples; affinities seeded as lift
- `tests/algorithms.test.ts` - new unit tests

Client

- `src/pages/admin/AdminRecommendationsPage.tsx` - "Lift" column and explanation
- `src/pages/admin/AdminInventoryPage.tsx` - ABC x XYZ matrix and Demand CV column
- `src/pages/admin/MarketAnalysisPage.tsx` - cluster panel shows the chosen k and silhouette
- `src/hooks/useAdmin.ts`, `src/types/index.ts` - matching types

Appendix B - running it

```
# unit tests
cd server && npm test

# regenerate demo data (wipes and reseeds; also rebuilds affinities as lift)
cd server && npm run db:seed

# rebuild only the recommendation model against existing orders (no data loss)
# admin -> Recommendations page -> "Rebuild model"
# or: POST /recommendations/admin/rebuild
```